

8-Puzzle Solver — Full Project Explanation

RA'26 Project — AI Search Algorithms

This document explains every part of the project so any team member can understand and present it.

Table of Contents

1. [What Is the 8-Puzzle?](#)
2. [State Representation](#)
3. [How We Generate Moves \(Successors\)](#)
4. [The Search Algorithms](#)
 - [A* Search](#)
 - [UCS \(Uniform Cost Search\)](#)
 - [Greedy Best-First Search](#)
 - [BFS \(Breadth-First Search\) — Bonus](#)
 - [DFS \(Depth-First Search\) — Bonus](#)
5. [Heuristic Functions](#)
 - [h1 — Misplaced Tiles](#)
 - [h2 — Manhattan Distance](#)
 - [h3 — Linear Conflict](#)
 - [Dominance Chain](#)
6. [Code Walkthrough](#)
7. [How to Run the Project](#)

1. What Is the 8-Puzzle?

The 8-puzzle is a sliding puzzle with a 3×3 grid containing tiles numbered 1–8 and one **blank space** (represented as `0`). The goal is to slide tiles into the blank space until you reach the **goal configuration**.

Start State:

1	2	3
4		6
7	5	8

→

Goal State:

1	2	3
4	5	6
7	8	

The challenge: **find the shortest (or cheapest) sequence of moves** to get from the start to the goal.

2. State Representation

We represent the 3×3 grid as a **flat Python list of 9 integers**:

```
# The grid is read left-to-right, top-to-bottom:
#   Position 0  1  2
#           3  4  5
#           6  7  8

GOAL = [1, 2, 3, 4, 5, 6, 7, 8, 0]  # 0 = blank
START = [1, 2, 3, 4, 0, 6, 7, 5, 8]  # blank is at position 4 (center)
```

Why a flat list? It's simple, easy to compare (`==`), and we can convert it to a `tuple` to store in a `set` for fast duplicate checking.

The Node Class

Each node in our search tree stores:

```

class Node:
    def __init__(self, value):
        self.value = value
        self.state = value           # the 9-element list
        self.parentstate = None      # pointer to parent Node (for path reconstruction)
        self.action = None           # "Up", "Down", "Left", "Right"
        self.edgeCost = 0            # cost of this single move (always 1)
        self.gOfN = 0                # g(n) = total cost from start to this node
        self.hOfN = 0                # h(n) = heuristic estimate to goal
        self.heuristicFn = None      # which heuristic function is used

```

3. How We Generate Moves (Successors)

The `get_successors(state)` function finds all valid moves from a given state.

Logic

1. **Find the blank** — `idx = state.index(0)`
2. **Convert to row, col** — `row, col = divmod(idx, 3)`
3. **Try all 4 directions** — Up, Down, Left, Right
4. **Check boundaries** — make sure the new position is inside the 3×3 grid
5. **Swap** — create a new state with the blank and the target tile swapped

```

def get_successors(state):
    successors = []
    idx = state.index(0)
    row, col = divmod(idx, 3)

    # Action names describe which direction the TILE moves (not the blank)
    moves = {
        'Down': (row - 1, col),    # blank goes up → tile above moves Down
        'Up': (row + 1, col),      # blank goes down → tile below moves Up
        'Right': (row, col - 1),   # blank goes left → tile at left moves Right
        'Left': (row, col + 1)     # blank goes right → tile at right moves Left
    }

    for action, (r, c) in moves.items():
        if 0 <= r < 3 and 0 <= c < 3:    # boundary check
            new_idx = r * 3 + c
            new_state = list(state)       # copy the state
            new_state[idx], new_state[new_idx] = new_state[new_idx], new_state[idx] # swap
            successors.append((action, new_state))

    return successors

```

Important Note About Directions

The **action names describe the tile's movement**, not the blank's. Example:

- If the blank is in the center and we swap it with the tile ABOVE → the tile moved **Down** into the center.

4. The Search Algorithms

All five algorithms share the **same core search loop** (in the `search()` method). The only thing that changes is **how they pick the next node to explore** — this is controlled by the **priority function $f(n)$** .

The General Search Pattern

1. Create a start node, add it to the frontier
2. While frontier is not empty:
 - a. Pick the "best" node from the frontier
 - b. If it's the goal → reconstruct the path and return
 - c. Mark it as explored
 - d. For each successor (neighbor):
 - If not explored, add to frontier with its priority
3. If frontier is empty → no solution

How We Reconstruct the Path

When we reach the goal, we follow the `parentstate` pointers back to the start:

```
curr = goal_node
while curr:
    path_actions.append(curr.action)    # e.g., "Down", "Right"
    path_states.append(curr.state)      # the full grid state
    curr = curr.parentstate             # go to parent
# Then reverse both lists (we traced backward)
```

4.1. A* Search

$$f(n) = g(n) + h(n)$$

Property	Value
f(n)	$g(n) + h(n)$ — total estimated cost
Complete?	Yes
Optimal?	Yes, if $h(n)$ is admissible (never overestimates)
Nodes expanded	Fewest among optimal algorithms
Memory	Moderate

How it works:

A* balances two things:

- **$g(n)$** = how much it cost to reach this node (actual cost so far)
- **$h(n)$** = estimated cost remaining to the goal (heuristic guess)

By adding both, A* prefers paths that are **cheap so far AND look promising ahead**. This is why it finds the optimal solution while exploring fewer nodes than UCS.

Code:

```
def Astar(self, heuristic_func=h2_manhattan):
    return self.search('Astar', heuristic_func)

# Inside search(), when mode == 'Astar':
f = next_node.gOfN + next_node.hOfN    #  $g(n) + h(n)$ 
```

4.2. UCS (Uniform Cost Search)

$$f(n) = g(n)$$

Property	Value
f(n)	$g(n)$ — path cost only

Complete? Property	Yes Value
Optimal?	Always
Nodes expanded	Most (expands many unnecessary nodes)
Memory	High

How it works:

UCS completely **ignores the heuristic** and only looks at the actual cost to reach each node. It always expands the cheapest-cost node first. It's basically Dijkstra's algorithm.

In the 8-puzzle, every move costs 1, so UCS behaves like BFS — but it's guaranteed optimal even with non-uniform costs.

Code:

```
def UCS(self):
    return self.search('UCS')

# Inside search(), when mode == 'UCS':
f = next_node.gOfN    # just g(n), no heuristic
```

4.3. Greedy Best-First Search

$$f(n) = h(n)$$

Property	Value
f(n)	$h(n)$ — heuristic only
Complete?	Not always
Optimal?	No
Nodes expanded	Fewest* (but may find suboptimal paths)
Memory	Low

How it works:

Greedy completely **ignores the path cost g(n)** and only looks at the heuristic — "how close does this state *look* to the goal?" It always expands the node that *appears* closest to the goal.

This makes it **fast** but **not optimal** — it can find a longer solution because it doesn't care about the cost already spent.

Code:

```
def Greedy(self, heuristic_func=h2_manhattan):
    return self.search('Greedy', heuristic_func)

# Inside search(), when mode == 'Greedy':
f = next_node.hOfN    # just h(n), no path cost
```

4.4. BFS (Breadth-First Search) — Bonus

Property	Value
Data Structure	FIFO Queue
Complete?	Yes
Optimal?	Yes (with unit costs)

Memory Property	High Value
-----------------	------------

How it works:

BFS explores all nodes at depth d before any node at depth $d+1$. Uses a regular list and pops from the **front** (FIFO = First In, First Out).

```
def BFS(self):
    return self.search('BFS')

# Inside search(), when mode == 'BFS':
current = frontier.pop(0)    # FIFO – take from front
frontier.append(next_node)   # add to back
```

4.5. DFS (Depth-First Search) – Bonus

Property	Value
Data Structure	LIFO Stack
Complete?	Not always (can loop infinitely)
Optimal?	No
Memory	Low

How it works:

DFS dives as deep as possible before backtracking. Uses a list and pops from the **back** (LIFO = Last In, First Out).

```
def DFS(self):
    return self.search('DFS')

# Inside search(), when mode == 'DFS':
current = frontier.pop()     # LIFO – take from back
frontier.append(next_node)   # add to back
```

5. Heuristic Functions

A **heuristic** is an educated guess of how far a state is from the goal. For A* to work correctly, the heuristic must be **admissible** — it must never overestimate the true cost.

5.1. h1 — Misplaced Tiles

Idea: Count how many tiles are NOT in their goal position (excluding the blank).

```
def h1_misplaced(state, goal):
    return sum(1 for s, g in zip(state, goal) if s != g and s != 0)
```

Example:

```
State:      Goal:
1 2 3      1 2 3
4 _ 6      4 5 6
7 5 8      7 8 _

Tile 5 is at position 7, should be at 4 → misplaced x
Tile 8 is at position 8, should be at 7 → misplaced x
h1 = 2
```

Why admissible? Each misplaced tile needs *at least* one move to reach its correct position. So counting misplaced tiles can never overestimate.

Complexity: $O(n)$ — very fast, just one pass through the list.

5.2. h2 — Manhattan Distance

Idea: For each tile, calculate the **horizontal + vertical distance** from its current position to its goal position. Sum all distances.

```
def h2_manhattan(state, goal):
    total = 0
    for i, tile in enumerate(state):
        if tile == 0:
            continue                # skip the blank
        goal_i = goal.index(tile)   # where should this tile be?
        current_row, current_col = i // 3, i % 3
        goal_row, goal_col = goal_i // 3, goal_i % 3
        total += abs(current_row - goal_row) + abs(current_col - goal_col)
    return total
```

Example:

```
State:      Goal:
1  2  3    1  2  3
4  _  6    4  5  6
7  5  8    7  8  _

Tile 5: position (2,1) → goal (1,1) → distance = |2-1| + |1-1| = 1
Tile 8: position (2,2) → goal (2,1) → distance = |2-2| + |2-1| = 1
All other tiles: 0
h2 = 2
```

Why admissible? Each tile must move at least its Manhattan distance — no shortcut exists. It can never overestimate.

Why better than h1? Manhattan gives a **tighter lower bound**. It counts exact distances, not just "misplaced or not". So $h2 \geq h1$ always.

5.3. h3 — Linear Conflict

Idea: Start with Manhattan distance, then add a penalty of **+2 for every pair of tiles that are in the same row/column as their goals but in the wrong order** (they must pass each other).

```
def h3_linear_conflict(state, goal):
    conflict_count = 0
    total = h2_manhattan(state, goal)    # start with Manhattan

    # Check each row for conflicts
    for row in range(3):
        row_tiles = [tiles in this row that belong in this row]
        for each pair (t1, t2) in row_tiles:
            if t1 is to the left of t2, but t1's goal is to the RIGHT of t2's goal:
                conflict_count += 1        # they must pass each other

    # Same check for each column
    ...

    return total + 2 * conflict_count    # +2 because passing requires 2 extra moves
```

Example of a linear conflict:

```
Row 0:  [3, 1, 2]    Goal row 0: [1, 2, 3]

Tile 3 is at column 0, goal is column 2
Tile 1 is at column 1, goal is column 0

Tile 3 is LEFT of tile 1, but tile 3's goal is RIGHT of tile 1's goal.
→ They must pass each other = 1 conflict → +2 moves
```

Why admissible? Each conflict adds exactly 2 extra moves (one tile must move out, then back). The Manhattan distance alone doesn't account for these conflicts.

5.4. Dominance Chain: $h_3 \geq h_2 \geq h_1 \geq 0$

This is a **key teaching point** of the project:

$$h_3 \geq h_2 \geq h_1 \geq 0$$

- **h1** just counts misplaced tiles (binary: right or wrong)
- **h2** measures actual distances (always $\geq h_1$ because a misplaced tile has distance ≥ 1)
- **h3** = h_2 + conflict penalty (always $\geq h_2$ because conflicts ≥ 0)

What does this mean for A?*

A more informed (higher) heuristic makes A* expand **fewer nodes**. So:

- A* with h_3 expands the **fewest** nodes
- A* with h_2 expands **more** nodes than h_3 but fewer than h_1
- A* with h_1 expands the **most** nodes

All three are admissible, so all give **optimal** solutions — but the stronger heuristic finds it faster.

6. Code Walkthrough

File Structure

Everything is in **one file**: SearchAlgorithms2 - Template .py

```
Lines 1-3      : Imports (heapq, streamlit, time)
Lines 4-14     : Node class
Lines 16-35    : get_successors() – move generation
Lines 37-46    : h1_misplaced(), h2_manhattan()
Lines 48-71    : h3_linear_conflict()
Lines 73-173   : SearchAlgorithms class (search engine)
Lines 175-195  : main() – console test (NOT changed from template)
Lines 197+     : Streamlit GUI code
```

The Search Engine — SearchAlgorithms.search(mode, heuristic_func)

This is the **core function**. Here's how it works step by step:

```

def search(self, mode, heuristic_func=h2_manhattan):
    # 1. Create the start node
    start_node = Node(self.start)
    start_node.gOfN = 0
    start_node.hOfN = heuristic_func(self.start, self.end)

    # 2. Initialize the frontier (priority queue or regular list)
    if mode in ['BFS', 'DFS']:
        frontier = [start_node]          # simple list
    else:
        f = ... # compute priority based on mode
        frontier = [(f, counter, start_node)] # min-heap

    explored = set() # states we've already visited

    # 3. Main loop
    while frontier:
        # Pick next node based on algorithm
        if mode == 'BFS':
            current = frontier.pop(0)    # FIFO
        elif mode == 'DFS':
            current = frontier.pop()     # LIFO
        else:
            _, _, current = heapq.heappop(frontier) # lowest f(n)

        # 4. Goal check
        if current.state == self.end:
            # Trace back to start via parentstate pointers
            # Return Path (actions), fullPath (states), totalCost
            ...

        # 5. Skip if already explored
        if tuple(current.state) in explored:
            continue
        explored.add(tuple(current.state))

        # 6. Expand – generate all successors
        for action, next_state in get_successors(current.state):
            if tuple(next_state) not in explored:
                next_node = Node(next_state)
                next_node.parentstate = current
                next_node.action = action
                next_node.gOfN = current.gOfN + 1
                next_node.hOfN = heuristic_func(next_state, self.end)

                # Add to frontier with priority
                if mode == 'UCS':    f = gOfN
                if mode == 'Astar': f = gOfN + hOfN
                if mode == 'Greedy': f = hOfN
                heapq.heappush(frontier, (f, counter, next_node))

    return [], [], -1 # no solution found

```

Why We Use `heapq`

`heapq` is Python's **min-heap** (priority queue). It always gives us the element with the **smallest priority** first. This is how A*, UCS, and Greedy always expand the "best" node.

The `counter` variable is used as a **tiebreaker** – when two nodes have the same $f(n)$, we expand the one added earlier (FIFO tiebreaking).

The Three Return Values

As required by the template:


```
return self.Path, self.fullPath, self.totalCost
```

Variable	Type	Description
Path	list[str]	Actions taken: ['Down', 'Right']
fullPath	list[list[int]]	All states from start to goal
totalCost	int	Number of moves (or -1 if no solution)

7. How to Run the Project

Prerequisites

```
pip install streamlit
```

Run the Console Tests (main function)

```
python3 "SearchAlgorithms2 - Template .py"
```

This runs the `main()` function and prints UCS, A*, and Greedy results to the terminal.

Run the GUI

```
streamlit run "SearchAlgorithms2 - Template .py"
```

This opens a browser with the interactive GUI where you can:

1. Enter any **start state** and **goal state**
2. Select an **algorithm** (A*, UCS, Greedy, BFS, DFS)
3. Choose a **heuristic** (h1, h2, h3) for A* and Greedy
4. See the **solution path** visualized step-by-step with colored tiles
5. See **live heuristic values** and the dominance chain check

Quick Comparison Table

Algorithm	f(n)	Optimal?	Complete?	Speed	When to use
A*	g+h	☑ Yes	☑ Yes	⚡ Fast	Best overall choice
UCS	g	☑ Yes	☑ Yes	🐢 Slow	When you have no heuristic
Greedy	h	☒ No	☒ Not always	⚡⚡ Fastest	When speed matters more than optimality
BFS	—	☑ Yes (unit cost)	☑ Yes	🐢 Slow	Simple, guaranteed
DFS	—	☒ No	☒ Not always	🔄 Varies	Low memory situations

Summary for Presentation

1. **The puzzle** is a 3×3 sliding tile game represented as a list of 9 integers.
2. **We implemented 5 search algorithms**, all sharing the same search loop — only the priority function differs.
3. **We implemented 3 heuristics** (h1, h2, h3) with increasing strength: $h3 \geq h2 \geq h1$.
4. *A with h3** is the best — it finds the optimal solution while expanding the fewest nodes.
5. **The GUI** lets you interactively test all combinations and visualize the solution.

